



SAPIENZA
UNIVERSITÀ DI ROMA

INFORMATION ENGINEERING, COMPUTER SCIENCE AND STATISTICS

CSMA/CA simulator

AUTONOMOUS NETWORKING

Professor:
Gaia Maselli

Student:
Lorenzo Sabatino, XXXXXXXXXX

Academic year 2025/2026

Contents

1	Introduction	5
1.1	Context	5
1.2	Motivations	5
1.3	Contributions	6
2	Design and Implementation	7
2.1	System Architecture	7
2.2	Communication Flow	9
2.3	Adaptive RL Policy	10
2.4	Performance Metrics	11
3	Experimental Results	13
3.1	Case Studies	13
3.2	Simulation Parameters	13
3.3	Performance Metrics	15
3.4	Simulations and Results	15
3.4.1	Node Density	15
3.4.2	Network Traffic	17
3.4.3	Learning Rate and Exploration	18
4	Conclusion	21
4.1	Limitations	21
4.2	Future Developments	21

Chapter 1

Introduction

The evolution of wireless communication technologies—from Wireless Sensor Networks (WSNs) to the Internet of Things (IoT)—has led to an increasing complexity in managing shared access to the transmission medium. This report describes the design and implementation of a simulator and it focuses on evaluating and improving the CSMA/CA (Carrier Sense Multiple Access with Collision Avoidance) protocol through the introduction of learning-based mechanisms.

1.1 Context

Wireless networks often operate in shared frequency bands where multiple nodes must contend for channel access. The standard CSMA/CA protocol, widely used in technologies like Wi-Fi, relies on a distributed backoff procedure to avoid collisions: when a node detects a busy channel, it waits for a random period determined by its Contention Window (CW).

However, in dynamic environments with varying traffic loads and node number, static contention window can lead to inefficiencies. If the CW is too small, the probability of collisions increases; if it is too large, the channel remains idle for long periods.

1.2 Motivations

The necessity of optimizing MAC protocols arises from the demand for autonomy in modern networks. An autonomous network should be capable of adapt its internal parameters in real-time based on environmental feedback rather than relying on human intervention or fixed rules.

Reinforcement Learning (RL) provides a robust tool for introducing this intelligence. By treating the selection of the contention window as a decision-making problem, nodes can learn to optimize their behavior to maximize throughput and minimize collisions.

1.3 Contributions

This work analyze and evaluate MAC protocols through a custom simulator. The main contributions are:

- A simulation environment that models node interactions, packet transmissions, and collision in a monitored area.
- Implementation of an adaptive mechanism where nodes use an ϵ -greedy strategy to pick their contention window from a set of available actions based on past rewards.
- A comparison between the standard CSMA/CA protocol against the RL-based adaptive approach across multiple simulation runs.

Chapter 2

Design and Implementation

This chapter describes the design and the implementation of the CSMA/CA simulation system. The system is designed to model a wireless environment where the agents coordinate their access to a shared medium.

2.1 System Architecture

The simulator is implemented as a time-driven system with four primary data structures that define the network's state:

- Packet represents the unit of data. It contains metadata such as the sender ID, destination ID, and creation time.
- Node contains the state machine, timers, and the Reinforcement Learning (RL) agent's local knowledge.
- PacketQueue manages buffered packets and is implemented as a linked list.
- Simulator is a global structure that holds the environment parameters, node list, statistics, etc.

The simulation follows a time-slotted approach where each iteration increments the simulation clock of $t=1$ (representing a `SLOT_TIME`). In each slot, the simulator updates the state of all nodes.

```
1 // Simulation loop.
2 for (t = 0; t <= horizon; t++) {
3     for (i = 0; i < nodeNum; i++) {
4         updateNode(nodes[i]);
5     }
6 }
```

This approach is well suited for MAC protocols where the timing of backoff slots, sensing periods, and frame durations are critical.

The simulation environment is a two-dimensional monitored area. At the beginning of each run, nodes are placed at random (x, y) coordinates.

For example, a 7×5 monitored area might be:

```
+-----+
| . . . 2 . . 6 |
| . 0 . . . . | The '.'
| 8 . 5 7 . . . | indicates
| . . . 9 . . . | the absence
| 1 . . . . 4 3 | of nodes.
+-----+
```

Node0 => [2, 8, 5, 7]

Node1 => [8, 5]

... => [...]

Node9 => [5, 7, 4]

Node5 and Node8 are neighbors of Node1.

Nodes are distributed within this space, and their communication is governed by their transmission range: two nodes can communicate only if their distance is $\leq \text{RANGE}$ and a collision occurs when a node is within range of multiple concurrent transmissions. Nodes perform physical carrier sensing by detecting incoming transmissions from neighbors to determine if the channel is busy.

Once the nodes are randomly positioned within the monitored area, the simulator creates the network topology by generating the neighbors lists: it scans the grid and identifies all nodes located within the specified RANGE of each other.

To represent the node behavior in the simulator, we define a Finite State Machine, as illustrated in Figure 2.1, where each state corresponds to a specific phase of the CSMA/CA protocol. The transitions are triggered by internal timers, by the status of the medium (carrier sensing), or by the arrival packets from the network. The system logic manages the transitions between the following states:

- **IDLE:** The node has no packets to send or is waiting for a packet generation event. It transitions to WAIT_DIFS if it has a packet in the output queue and the channel is sensed idle. If a DATA packet starts arriving from a neighbor, it transitions to RECEIVING.
- **BACKOFF:** The node has a packet to transmit and is decrementing its backoff counter while the channel is sensed idle. When the counter reaches zero, it transitions to TRANSMITTING to send its DATA packet.
- **TRANSMITTING:** The node is currently sending a DATA or ACK packet. Upon completion, if the packet was DATA, the node moves to WAIT_ACK. If it was an ACK, it returns to IDLE.
- **RECEIVING:** The node is receiving a DATA or ACK packet. If a DATA packet is successfully received, the node transitions to WAIT_SIFS to prepare an acknowledgement. Otherwise it returns to IDLE.
- **WAIT_ACK:** After transmission, the node waits, for a given period of time, an ACK packet from the receiver. If a valid ACK is detected, it transitions to RECEIVING. If the ACK_TIMEOUT is hit, it assumes a collision or loss has occurred and, it returns to IDLE to schedule a retransmission. (Here it also updates the Contention Window or RL values.)

- **WAIT_DIFS and WAIT_SIFS:** Waiting periods required by the CSMA/CA protocol. After DIFS, if the channel becomes busy, it returns to IDLE to defer transmission. If the timer expires while the channel is idle, it moves to BACKOFF. After SIFS, once the timer expires, the node transitions to TRANSMITTING.

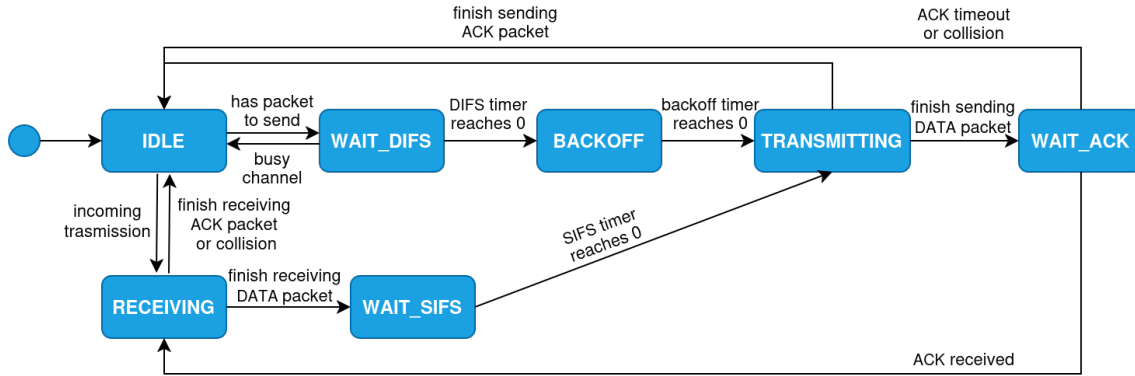


Figure 2.1: Finite State Machine of a node in the CSMA/CA simulator.

2.2 Communication Flow

The communication process follows the standard CSMA/CA protocol. The flow of a single transmission attempt is:

- **Packet generation:** In each time slot, a node generates a DATA packet with a probability `GEN_PROB` and places it in its output queue.
- **Carrier sensing:** The node performs a physical carrier sense by checking if any neighbors are currently in the TRANSMITTING state. If the channel is idle, it waits for a DIFS period. If the channel becomes busy during this time, the process restarts.
- **Backoff:** Nodes pick a random timer from their Contention Window (CW). This backoff counter decrements only when the channel is idle.
- **Data transmission:** When the backoff counter reaches zero, the node broadcasts its DATA packet to all neighbors. Here we model a half-duplex system: if a node is TRANSMITTING, it cannot "hear" incoming packets and will automatically discard any arrivals.
- **Acknowledgment (ACK):** Upon successful reception of a DATA packet, the destination waits for a SIFS interval and then sends an ACK packet back to the sender. A transmission is considered successful only if the sender receives this ACK before a timeout; otherwise, it doubles its CW and attempts a retransmission.

Each node manages its internal data flow using two distinct packet queues:

- **Output Queue:** This acts as a buffer for packets the node wants to send. It stores locally generated DATA packets and high-priority ACK responses. ACKs are moved to the front of this queue to ensure they are sent immediately after the SIFS period.
- **Input Queue:** This acts as a reception buffer where the network delivers packets. During each slot, the simulator checks this queue. If it contains multiple packets simultaneously, it indicates a collision. If only one packet is present, the node begins the RECEIVING process.

To avoid the computational cost of deep-copying packets for every neighbor, the simulator uses a reference counting mechanism. As illustrated in Figure 2.2, when a node sends a packet, it doesn't create n copies for n neighbors. Instead, it creates one packet object and gives each neighbor a reference to it.

For every neighbor that successfully adds the packet to its input queue, the packet's `refcount` is increased by one. Once a node is done with the packet (either by processing it or discarding it due to a collision), it calls `releasePacket`, which decreases the count. The memory occupied by the packet is only actually deleted from the system when the `refcount` reaches zero, meaning that no more nodes are using it.

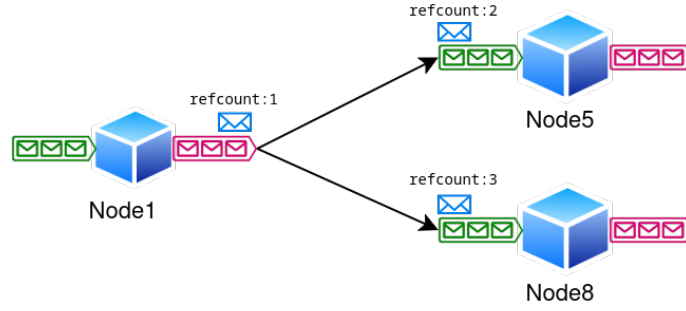


Figure 2.2: Representation of the communication flow between nodes.

2.3 Adaptive RL Policy

The core innovation of this simulator is the transition from the static contention window of standard CSMA/CA to an Adaptive RL-based policy. This approach treats the selection of the contention window as a decision-making problem, allowing nodes to optimize their behavior in real-time based on environmental feedback.

- **Actions:** The agent selects from a set of K arms, where each arm corresponds to a specific CW value (e.g., $CW \in \{15, 31, \dots, 1023\}$).
- **Reward function:** After every transmission attempt, the environment provides a reward. A successful transmission (ACK received) yields a positive reward, while a collision or time-out results in a negative reward.

- **Update rule:** The agent maintains an estimated value (Q-value) for each arm. These are updated using an incremental update rule with a constant step-size α , allowing the node to prioritize recent network conditions over old data:

$$Q(A) \leftarrow Q(A) + \alpha[\text{Reward} - Q(A)]$$

- **Policy:** An ϵ -greedy strategy is used to balance exploration (trying new CW values) and exploitation (using the best CW value found so far).

```

1  /* Select the next contention window using the Epsilon-Greedy strategy. */
2  void selectAction(Node *node) {
3      // Explore. Choose a random arm.
4      if (rand() < epsilon) {
5          action = rand() % ARMS_NUM;
6      }
7      // Exploit. Choose the arm with the highest Q-value.
8      else {
9          int bestArm = 0;
10         double maxQ = node->qValues[0];
11         for (int i = 0; i < ARMS_NUM; i++) {
12             if (node->qValues[i] > maxQ) {
13                 maxQ = node->qValues[i];
14                 bestArm = i;
15             }
16         }
17         action = bestArm;
18     }
19 }

```

2.4 Performance Metrics

To evaluate the efficiency of the autonomous RL policy against the standard protocol, the simulator tracks global performance metrics. These metrics are computed at the end of each simulation run:

- **Packet Delivery Ratio (PDR):** Measures the reliability of the network by calculating the percentage of generated packets that are successfully delivered and acknowledged. It is computed as:

$$PDR = \frac{\text{PacketsAcked}}{\text{PacketsGenerated}}$$

A higher PDR indicates a robust protocol that effectively manages contention and minimizes packet loss.

- **Throughput:** Represents the effective data rate of the network, calculated as the number of successfully received packets over the total simulation time.

$$Throughput = \frac{PacketsReceived}{TotalSimulationTime(ticks)}$$

Throughput reflects how efficiently the medium is utilized; the RL policy aims to maximize this metric by selecting optimal Contention Windows that reduce idle time.

- **Collision Rate:** Identifies the impact of simultaneous transmissions. It is the ratio of detected collisions to the total number of packets sent.

$$CollisionRate = \frac{Collisions}{PacketsSent}$$

A collision is recorded when a node is within range of multiple concurrent transmissions. Reducing this rate is a primary goal of the RL agent's backoff strategy.

- **Average Latency:** This tracks the time elapsed from the moment a packet is generated to the moment it is successfully acknowledged.

$$AvgLatency = \frac{\sum(TimeofACKReception - PacketCreationTime)}{TotalPacketsAked}$$

Because our wireless environments is stochastic due to random node placement and randomized backoff timers, the simulator performs multiple independent runs. Then each performance metric tracks the mean and variance across these runs to allow for comparison between the "Standard" and the "Adaptive CW" models.

Chapter 3

Experimental Results

This chapter presents the experimental analysis of the CSMA/CA simulator, comparing the performance of the standard protocol with the adaptive approach based on Reinforcement Learning.

3.1 Case Studies

Here we test two configurations:

- **Standard CSMA/CA:** In this baseline protocol, nodes operate using a Binary Exponential Backoff mechanism. When the medium is sensed busy, a node waits for a DIFS period and then selects a random backoff timer within the range $[0, CW - 1]$. The CW starts at a minimum value (CW_MIN) and doubles after each collision until it reaches a maximum threshold (CW_MAX).
- **Adaptive CW:** In this model, each node acts as an RL agent that treats the selection of the Contention Window as a Multi-Armed Bandit problem. The node learns to pick the optimal CW size from a set of "arms" based on the rewards received.

The motivation behind this choice is to allow nodes to learn the optimal CW size for the current network density. If a node experiences frequent collisions, it will learn to pick larger windows. Instead, if a node receives few transmissions (low traffic in the network), it will learn to pick a smaller window in order to maximize the rewards received over time.

3.2 Simulation Parameters

The simulation parameters, summarized in Table 3.1, are configured to reflect a typical WSN scenario.

To test the policies, we simulate the network under varying conditions:

Parameter	Value	Description
HORIZON	5,000,000	The total duration of a single simulation run.
SIM_RUNS	20	Number of times the simulation is executed. The results are averaged across these runs.
SEED	-1	Controls the random number generator seed. -1 enables a random seed for each simulation run.
DATA_DUR ACK_DUR	4 1	The transmission time for data packets and acknowledgments.
SIFS_DUR DIFS_DUR	2 4	Short Inter-Frame Space (SIFS) and Distributed Inter-Frame Space (DIFS) durations.
ACK_TIMEOUT	8	The maximum time a node waits for an ACK before assuming a collision or loss occurred.
NODE_NUM	15	The number of nodes in the network competing for the medium.
RANGE	3	Transmission range of each node within the monitored area.
QUEUE_CAPACITY	50	Maximum number of packets that can be stored in a node's internal queue. When the queue is full, newly generated packets are dropped.
GEN_PROB	0.003	The probability that a node generates a new packet in a given tick.
CW_MIN CW_MAX	15 1023	Lower and upper bounds of the contention window used for random backoff.
SLOT_TIME	1	The duration of a single backoff slot, which corresponds to the system's atomic tick.
RL_EPSILON	0.10	The exploration rate for the ϵ -greedy policy. It defines the probability of choosing a random action.
RL_ALPHA	0.30	Controls how fast the learning agent updates its knowledge. It allows the agent to adapt to changing network conditions.
ARMS_NUM	7	The number of actions available to the RL agent. Each arm represents a different CW the agent can choose from.
MAREA_WIDTH MAREA_HEIGHT	10 10	Define the width and height of the monitored area.

Table 3.1: Simulation paramters.

- **Number of Nodes (NODE_NUM):** We scale the network from a low-density (5 nodes), medium-density (15 nodes) to a high-density scenario (30 nodes) to observe the impact on collision rates.
- **Packet Generation Probability (GEN_PROB):** High values simulate a saturated network where nodes almost always have packets to send. We start from sending occasional updates ($\text{GEN_PROB} = 0.001$), standard load (0.003), to saturated medium where nodes always have packets in the queue (0.005).
- **Exploration Rate (RL_EPSILON):** With low ϵ (0.01) the node quickly converges to a strategy but may get stuck in a sub-optimal CW if the environment changes. Medium ϵ (0.10) represents the standard balance between exploration and exploitation. With high ϵ (0.30) the node remains flexible and learns faster in dynamic conditions.

3.3 Performance Metrics

To quantify the efficiency of the protocols, we track the following metrics:

- **Packet Delivery Ratio (PDR):** The ratio of successfully acknowledged packets to the total packets generated.
- **Throughput:** The average number of packets successfully received per time unit.
- **Average Latency:** The average time elapsed between the generation of a packet and its successful delivery.
- **Collision Rate:** The ratio of collisions to the total number of transmission attempts.

3.4 Simulations and Results

3.4.1 Node Density

The first scenario evaluates how the system scales as the number of nodes increases (5, 10, and 15 nodes). In a high-density environment, the probability of multiple nodes choosing the same backoff slot increases, leading to higher collision rates and lower efficiency.

The results showed in Table 3.2 clearly show that the Standard CSMA/CA performs slightly better in terms of throughput and collision rate in low-density scenarios. This is because the Standard protocol immediately resets the contention window (CW) to CW_{min} after a successful transmission, allowing for faster subsequent access. Adaptive RL, by contrast, maintains a memory of rewards and might “explore” larger CWs or converge to a value larger than CW_{min} to avoid potential collisions, which also adds a small latency penalty.

However, as density increases, the Standard protocol’s performance decreases and the Adaptive CW maintains a high PDR and significantly lower latency by learning to avoid collisions

Nodes	Protocol	PDR	Throughput	Avg Latency	Collision Rate
5 (Low)	Standard	99.93%	0.0314	53.65	4.57%
	Adaptive	99.72%	0.0200	97.43	6.72%
10 (Med)	Standard	84.57%	0.0778	386.46	14.46%
	Adaptive	98.65%	0.0482	182.32	10.31%
15 (High)	Standard	78.93%	0.0916	768.10	23.46%
	Adaptive	95.57%	0.0722	351.27	17.77%

Table 3.2: Experimental results on variable node density.

through better CW management. This occurs because the Standard protocol's logic is too aggressive and nodes repeatedly collide at CW_{min} before backing off again. The Adaptive protocol maintains a much higher PDR because the RL agent learns that in high-density environments, a larger CW is the "optimal arm" to minimize collisions.

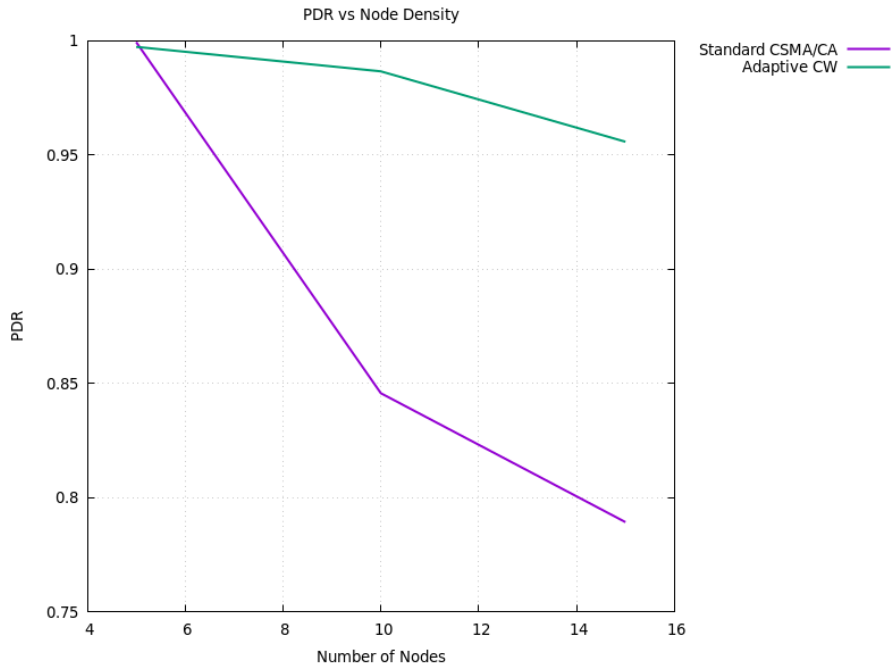


Figure 3.1: PDR comparison across node densities.

This shows us how fixed policies are not scalable: the Standard protocol is greedy and fast in low density networks. Instead, the Adaptive approach avoids the congestion by selecting higher contention windows, without waiting for a series of collisions, in higher density networks.

3.4.2 Network Traffic

We also evaluated the system under different traffic loads by varying the packet generation probability.

Gen. Prob.	Protocol	PDR	Throughput	Avg Latency	Collision Rate
0.003 (Low)	Standard	99.97%	0.0491	68.65	6.52%
	Adaptive	99.98%	0.0294	68.33	9.13%
0.005 (Med)	Standard	95.93%	0.0752	280.27	13.83%
	Adaptive	98.94%	0.0473	161.91	8.55%
0.010 (High)	Standard	70.44%	0.0812	640.49	19.39%
	Adaptive	78.30%	0.0776	584.19	17.61%

Table 3.3: Experimental results on variable network traffic.

With low traffic both protocols perform well, with PDR above 99.9%. In high traffic scenarios both the standard and the adaptive PDRs collapse.

Here the Adaptive protocol try to balance the latency-throughput trade-off: it prefers to wait longer to ensure a successful delivery rather than attempting frequent transmissions that lead to collisions.

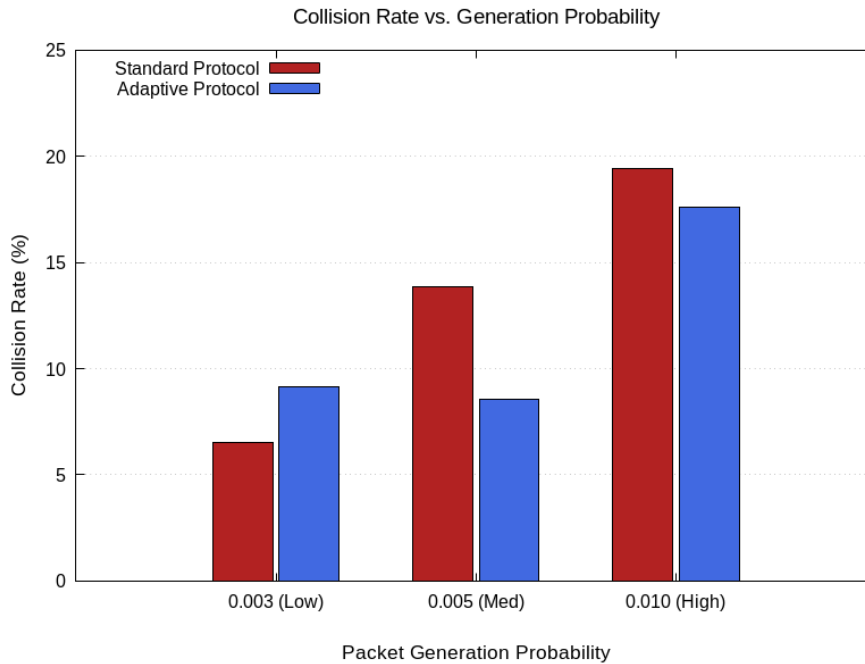


Figure 3.2: Collision rates under varying traffic.

3.4.3 Learning Rate and Exploration

Finally, we analyzed the impact of the RL hyperparameters, α (alpha) and ϵ (epsilon), on the Adaptive CW performance.

α	Protocol	PDR	Throughput	Avg Latency	Collision Rate
0.10	Adaptive	98.74%	0.0470	175.41	9.63%
0.20	Adaptive	96.43%	0.0467	267.14	10.45%
0.30	Adaptive	94.53%	0.0445	398.46	9.30%

Table 3.4: Experimental results on variable learning rate.

ϵ	Protocol	PDR	Throughput	Avg Latency	Collision Rate
0.001	Adaptive	97.94%	0.0486	199.37	11.03%
0.010	Adaptive	97.97%	0.0459	209.69	9.10%
0.030	Adaptive	98.17%	0.0476	197.27	13.84%

Table 3.5: Experimental results on variable exploration.

Testing α values showed that a lower learning rate provided the best PDR. A higher α makes the agent more reactive to changes in network conditions but also increased the probability of instability. With high α values a single random collision might cause the node to immediately shift its Q-values toward a very large CW, even if the network isn't actually congested, causing unnecessary delays.

Increasing ϵ showed that moderate exploration is better. The highest PDR and lowest latency were achieved with $\epsilon=0.030$, where the agent frequently checks other CW sizes to adapt to changing network conditions, but at cost of more collisions.

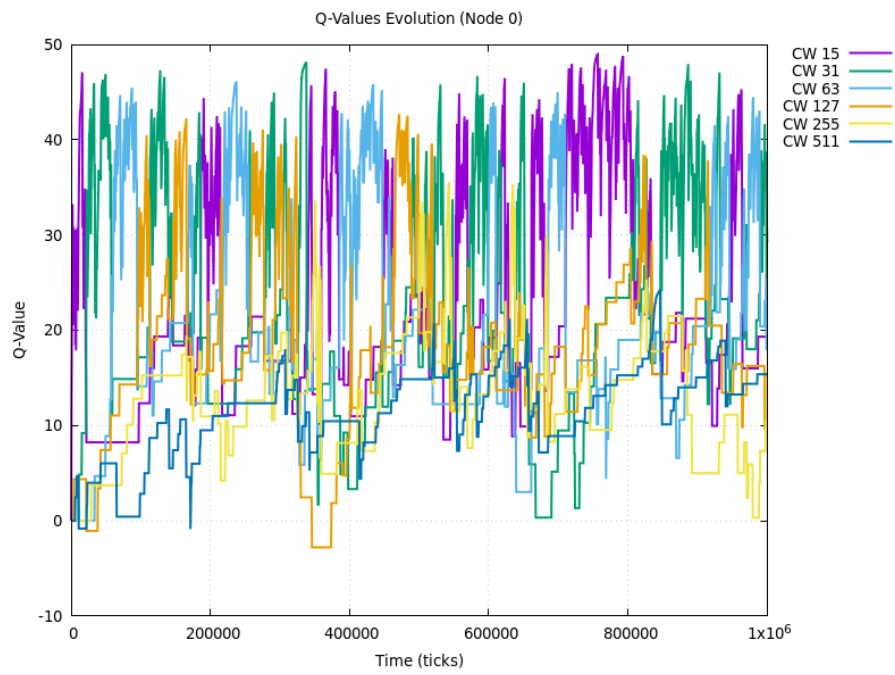


Figure 3.3: Evolution of Q-values over time.

Chapter 4

Conclusion

This work presented the design and implementation of a time-driven simulator for the CSMA/CA protocol, focusing on the optimization of the Contention Window (CW) through autonomous Reinforcement Learning techniques. The analysis compared a standard backoff approach against an adaptive agent using an ϵ -greedy bandit algorithm. The experimental results highlighted how the introduction of autonomy allows the network to better adapt to varying node densities and traffic conditions.

4.1 Limitations

The current version of the simulator introduces some simplifications that should be addressed to achieve a more realistic simulation:

- **Pseudorandom Generation:** The simulator utilizes the standard C `rand()` function for backoff counter selection and node placement. These functions are not cryptographically strong and that can bias the stochastic behavior of backoff selections and node positioning in very long simulation run.
- **Simplified Physical Layer:** The communication model permits a transmissions between nodes if they are within a fixed RANGE. This ignores realistic propagation effects such as path loss and multi-path, which are critical in urban environments.
- **Static Topology:** In the current implementation, nodes are assigned random coordinates at the start but remain stationary throughout the simulation. This prevents the analysis of dynamic topology changes like in dronet.

4.2 Future Developments

Starting from the results achieved, several extensions can be implemented to enhance the simulator's depth and realism:

- **Advanced Reinforcement Learning:** Future versions could replace the ϵ -greedy bandit with more advanced algorithms like Q-Learning. These would allow the nodes to consider a larger state space (e.g., current queue size, estimated number of neighbors) rather than just immediate rewards.
- **3D Environment:** To better simulate Unmanned Aerial Vehicles (UAVs), the 2D grid could be expanded into a 3D grid.
- **Energy Consumption Analysis:** For autonomous sensors and drones, battery life is as critical as throughput. Future versions should include an energy model that tracks the cost of sensing, transmitting, and receiving.